Reflexive-Core: Single-Context Metacognitive Security for Agentic LLMs

Alex Stanton

Independent Researcher

alex@thinkpurple.io

https://linkedin.com/in/alexlstanton

Date: October 2025

Status: Research Prototype

Abstract

This work presents Reflexive-Core, an evolutionary step in A2AS (Agent-to-Agent Security) implementation that moves from passive security markup to structured in-context metacognitive reasoning. A2AS was developed to eliminate multi-pass verification overhead (commonly requiring multiple separate LLM calls, often 5 or more per request). A2AS provides `<a2as:defense>` and `<a2as:policy>` primitives that establish security boundaries before user context arrives, but without structured metacognitive reasoning, these remain passive annotations the model may inconsistently follow. Early implementations, such as Christian Posta's agentgateway, validate A2AS operational viability by enforcing wrapper structure and enabling deterministic output validation before results return to agents.

Reflexive-Core demonstrates the next evolution: **leveraging the sophisticated security reasoning that every frontier LLM inherently possesses** by structuring in-context inference into specialized sub-personas (Preflight Analyst, Security Analyst, Controlled Executor, Compliance Validator). Rather than relying on passive markup, each persona actively reasons about threats, policies, and risks within the context window, transforming A2AS from unstructured guidance into intelligent runtime security governance.

This architecture is grounded in recent evidence for measurable metacognitive capabilities in frontier LLMs (r=0.2-0.3 confidence-behavior correlations, per Ackerman 2025), the cognitive synergy demonstrated by Solo Performance Prompting (+7.1% to +18.5% accuracy gains on GPT-4, Wang et al. 2023), and Constitutional AI's principle-based self-critique.

Token overhead analysis shows 1.9× cost for small contexts (1K tokens) but scales to <1.01× for realistic production contexts (50K+ tokens)—economically viable compared to both external enforcement overhead and multi-pass verification (2-3×).

Reflexive-Core is positioned not as a replacement for external systems but as complementary evolution: deterministic brokers (agentgateway-style) handle wrapper enforcement and output validation while in-context personas provide contextual intelligence. This hybrid approach—external reliability plus internal reasoning—represents a possible future of A2AS implementation. This work contributes a working implementation specification and empirical token analysis, demonstrating that structured metacognitive reasoning can operationalize security intelligence that passive markup approaches leave untapped.

Following vendor guidance (Anthropic, OpenAI, Google Vertex AI, Azure OpenAI), we structure prompts with XML-delimited sections; **paired with schema-bound generation for each sub-persona**, this yields well-formed, parseable outputs consistent with recent XML-prompting theory and supports our hypothesis that XML-schema–scaffolded sub-personas **improve metacognitive self-governance** (higher parse validity and phase-order adherence).

Keywords: LLM Security, Reflexive Review, Metacognition, Sub-Personas, Agentic AI, A2AS, Prompt Injection Defense, Policy Enforcement, Constitutional AI, agentgateway

1. Introduction

The rapid deployment of autonomous AI agents introduces unprecedented security challenges. Unlike traditional software with well-defined execution boundaries, large language model (LLM) agents operate through natural language reasoning, making them vulnerable to prompt injection attacks, role confusion, and unintended privilege escalation. Existing security approaches typically employ external verification loops requiring multiple separate LLM calls (often 5 or more per request, sometimes exceeding 15 in complex topologies), introducing latency, state management complexity, and expanded attack surfaces—or rely on hardened system prompts that prove brittle against sophisticated adversarial inputs.

The A2AS (Agent-to-Agent Security) framework was developed specifically to address these limitations. A2AS provides structured primitives— `<a2as:defense>` for input validation, `<a2as:policy>` for declarative permissions, behavior certification—that enable **single-context security reasoning** without external dependencies. However, A2AS is a framework, not a full implementation. It defines the *what* (security boundaries, policy markup) but leaves the *how* (operationalizing these primitives) to developers.

Reflexive-Core operationalizes A2AS's vision by implementing the missing piece: **metacognitive security reasoning through structured multi-persona analysis**. Rather than treating A2AS markup as passive annotations and *hoping* the model will adhere, Reflexive-Core makes them **active gover-**

nance mechanisms executed within a single model inference. The framework partitions reasoning into specialized analytical phases—threat detection, policy enforcement, execution, and compliance validation—each with explicit checkpoints and fail-closed defaults, creating a self-governing security routine that requires no external verification loops.

XML-delimited sections to separate instructions, examples, and outputs, following vendor guidance across Anthropic, OpenAI, Google Vertex AI, and Azure OpenAI. Recently, research from Alpay, F. & Aplay T. [XML Prompting as Grammar-Constrained Interaction: Fixed-Point Semantics, Convergence Guarantees, and Human-AI Protocols] formalizes XML prompting as **grammar-constrained interaction**, providing well-formedness guarantees under an XML schema. The hypothesis is that XML-schema–scaffolded sub-personas (one context window) **increase the odds of successful metacognitive self-governance**, operationalized as higher schema validity and phase-order adherence, relative to unstructured prompts. [Anthropic; OpenAI; Vertex AI; Azure; Alpay & Alpay, 2025]

1.1 Motivation: From Passive Markup to Active Metacognitive Reasoning

The A2AS framework was created to eliminate the operational burden and security risks of multi-pass verification architectures (commonly requiring multiple separate LLM calls per request). Initial implementations, such as Christian Posta's agentgateway, validated A2AS's operational viability through wrapper enforcement—middleware that ensures A2AS markup structure is correctly applied and enables deterministic validation of outputs before results return to agents. Posta's work demonstrated that A2AS primitives can successfully mitigate prompt injection attacks in production environments today, providing critical proof-of-concept for the framework's viability.

However, A2AS primitives alone face inherent limitations:

1. **Passive guidance without active reasoning:** `<a2as:defense>` and `<a2as:policy>` provide the LLM with security boundaries before user context arrives, but without structured metacognitive reasoning, these remain passive annotations the model must follow implicitly
2. **Underutilizes LLM capabilities:** Frontier models possess sophisticated reasoning about threats, policy compliance, and security implications—capabilities left untapped without explicit cognitive scaffolding
3. **Inconsistent adherence:** Models "forget" constraints during complex reasoning, fail to detect subtle attacks, and inconsistently apply policies without structured enforcement

The Key Insight: Every frontier LLM inherently possesses cybersecurity reasoning capabilities—the ability to understand threats, evaluate policy implications, assess risk, and detect manipulation. These capabilities emerge from training on vast corpora including security documentation, threat analysis, and adversarial examples. A2AS primitives provide "sudo allow/deny lists" to the LLM, but without structured metacognitive reasoning, the model treats them as unstructured guidance rather than actively enforced security architecture.

Reflexive-Core's Thesis: Security enforcement can evolve beyond passive markup by **leveraging the LLM's built-in cybersecurity capabilities through structured in-context reasoning**. Rather than hoping the model maintains awareness of A2AS primitives, the framework partitions inference into specialized security personas that actively reason about threats, policies, and risks within the context window.

Three factors make this evolution both necessary and viable:

1. **agentgateway's Validation:** Posta's work proves A2AS primitives are operationally sound. The focus can now shift to optimizing enforcement mechanisms.
2. **Emergent but Limited Metacognition:** Ackerman (2025) demonstrates frontier LLMs exhibit measurable introspective abilities—but these remain modest (r=0.2-0.3 confidence-behavior correlations). Token probability signals suggest "upstream internal signals" enabling metacognition, yet models show "graded and inconsistent" performance with "variability across datasets/modes." This motivates explicit governance structures rather than relying on unstructured "be careful" prompts.
3. **Production Constraints:** Real-time applications cannot tolerate external processing delays. In-context reasoning eliminates round-trip latency entirely.

1.2 Contributions

This work makes the following contributions:

- **Evolutionary Step Beyond Passive Markup:** Building on agentgateway's validation that A2AS primitives work operationally, Reflexive-Core demonstrates the next evolution—structured in-context security reasoning that leverages LLMs' inherent cybersecurity capabilities.
- **Shift from Passive to Active Security:** The framework transforms A2AS from unstructured guidance (passive annotations the model should follow) to structured metacognitive reasoning (specialized sub-personas that actively enforce security through explicit analysis).
- **Metacognitive Architecture Pattern:** A phase-ordered framework that attempts to invoke structured metacognitive responses through explicit cognitive scaffolding, achieving pseudo-determinism through controlled structure despite the inherent non-determinism of LLM outputs.
- **Empirical Validation:** Token overhead analysis shows 1.9× cost for small contexts, scaling to <1.01× for production workloads—proving that in-context intelligence is economically viable.
- **Working Implementation Specification:** Production-ready XML schema demonstrating how to operationalize A2AS through structured persona reasoning.
- **Future Integration Path:** Design compatible with deterministic brokers (agentgateway-style)—external systems handle wrapper enforcement and output validation while in-context personas provide intelligent runtime security reasoning.

1.3 Scope and Non-Goals

In Scope: Single-context reasoning for read-only, security-sensitive operations (email summarization, document analysis, information retrieval). Applicable to frontier models with 100K+ context windows (Claude Opus 4+, GPT-4.1+, Gemini 2.5 Pro+, Grok 4+).

Out of Scope: Network-level security, tool authorization frameworks, multi-agent orchestration, write operations, or replacement for external security controls. Reflexive-Core is intentionally designed as **one layer in defense-in-depth**, not a complete security solution.

2. Background and Related Work

2.1 Agent-to-Agent Security (A2AS)

The A2AS framework, developed by collaborators from OWASP, Google, Salesforce, AWS, and other organizations, was created specifically to eliminate the operational overhead and security risks of external verification architectures. Traditional agentic security approaches require multiple separate LLM calls per request (often 5 or more, sometimes exceeding 15 in complex topologies with policy check → threat scan → execution → verification → compliance), introducing latency, state management complexity, and expanded attack surfaces.

A2AS provides structured primitives that enable **single-context security:**

- `<a2as:defense>` establishes security boundaries between system instructions and external content
- `<a2as:policy>` declares permissions and constraints in natural language
- Behavior certification mechanisms for agent-to-agent trust

Current Implementation Approaches: A2AS is a **framework**, not a full implementation specification. It defines what security boundaries should exist but leaves enforcement mechanisms to implementers. Two implementation strategies have emerged:

1. **Wrapper Enforcement (agentgateway, Christian Posta):** Middleware sits between agent and LLM, ensuring A2AS markup structure is correctly applied to requests and enforcing deterministic validation of outputs before results return to agents. This approach **validates that A2AS primitives work operationally**—it's a critical proof-of-concept demonstrating A2AS viability in production environments.

   Reference: Posta, C. (2024). How to Mitigate Prompt Injection Attacks with A2AS and agentgateway. *LinkedIn*. https://www.linkedin.com/pulse/mitigate-prompt-injection-attacks-a2as-agentgateway-christian-posta-tmaxc/

2. **Lightweight Markup:** Developers include A2AS tags in system prompts, relying on the model's implicit understanding to maintain security boundaries.

The Limitation: Both approaches underutilize a critical capability: **LLMs inherently possess sophisticated reasoning abilities about security, policy, and threat detection**. A2AS primitives provide "sudo allow/deny lists" and security guidance to the LLM, but without structured metacognitive reasoning, these remain passive annotations the model must follow implicitly. Neither approach leverages the model's capacity for nuanced, actively-reasoned security enforcement.

Reflexive-Core's Contribution: This work proposes an evolutionary step beyond passive markup: **operationalize A2AS through structured metacognitive reasoning that runs in-context**. Rather than treating `<a2as:policy>` as guidance the model should follow, Reflexive-Core transforms it into an **active governance mechanism** enforced through specialized sub-personas that leverage the LLM's inherent cybersecurity capabilities.

Future Vision: A deterministic broker (agentgateway-style) could manage A2AS+Reflexive together—using external systems for wrapper enforcement and output validation while delegating intelligent, runtime security reasoning to in-context personas. This hybrid approach combines the operational reliability of structural enforcement with the contextual intelligence of metacognitive reasoning.

Reference: https://www.a2as.org/

2.2 Metacognition in LLMs

Metacognition—the ability to monitor and control one's own cognitive processes—is fundamental to self-governed security. Recent empirical work establishes both the promise and limits of LLM metacognition:

Ackerman (2025) introduced non-linguistic behavioral paradigms (inspired by animal cognition research) to test LLM metacognition without relying on self-reports. Key findings:

- Frontier models (GPT-4.1, Claude Opus 4) show **partial correlations (r=0.2-0.3)** between internal confidence (token probabilities) and delegation decisions
- Metacognition is "graded and inconsistent...variability across datasets/modes"
- Recent models show "increasingly strong evidence" vs. mid-2024 models
- Token probability analysis suggests "upstream internal signals" enabling introspection
- Even with strong teammates, models often over-answer or mis-delegate; only Sonnet 3.5 nudged team accuracy above max(self, teammate) (+5.4/+8.6 points)

Critical Implication: Metacognition exists but is weak. Reliance on freeform "be secure" prompts is insufficient. **Explicit structures** are needed that harvest limited capabilities and fail safely when absent.

Citation: Ackerman, C. (2025). Evidence for Limited Metacognition in LLMs. *arXiv preprint arXiv:2509.21545*. https://arxiv.org/abs/2509.21545

2.3 Solo Performance Prompting (SPP)

Wang et al. (2023) demonstrated that **cognitive synergy emerges** when a single LLM adopts multiple specialized personas:

SPP shows consistent gains vs Standard prompting on GPT-4:

- +7.1% (Trivia Creative Writing, N=5)
- +10.0% (Trivia Creative Writing, N=10)
- +4.8% (Codenames Collaborative)
- +18.5% (Logic Grid Puzzle)

Chain-of-Thought is flat/negative on knowledge tasks. These are average scores across runs with and without a system message (Table 2, Wang et al., 2023).

Additional findings:

- Reduced hallucination through multi-perspective critique
- **Only emerges in frontier models** (GPT-4+, not GPT-3.5)
- Each persona must have **distinct, non-overlapping expertise**

Key Insight: Vague "think like an expert" prompts fail. Personas need specific, differentiated responsibilities to create genuine cognitive diversity.

Application to Security: Rather than one "security-aware" agent, Reflexive-Core implements threat detection, policy enforcement, execution, and compliance validation as separate personas that critique each other.

Note: While SPP benchmarks measure task performance (not security outcomes directly), the underlying mechanism—cognitive diversity through perspective-taking—theorizes strong transfer to cybersecurity sub-persona tasks.

Citation: Wang, Z., et al. (2023). Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration. *arXiv preprint arXiv:2307.05300*. https://arxiv.org/abs/2307.05300

2.4 Constitutional AI

Bai et al. (2022) showed models can self-critique and self-revise according to written principles:

- Models generate critiques of their own outputs
- Revisions improve alignment with constitutional principles
- Achieves safer behavior without human feedback on harmful examples
- Scales oversight beyond case-by-case human review

Application: Each Reflexive-Core persona applies constitutional principles (authenticity, least privilege, transparency, privacy, harm prevention) from its specialized perspective.

Citation: Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*. https://arxiv.org/abs/2212.08073

2.5 Related Security Frameworks

Multi-Pass Verification: Tools like LangChain's Constitutional Chain use separate LLM calls for generation and critique. While effective, this approach incurs 2-3× token overhead and introduces state management complexity.

Prompt Hardening: Defensive prompts attempt to "train" the model against injection. However, sophisticated attacks can override these instructions, and hardening often reduces model usefulness.

External Guardrails: Services like Anthropic's Claude Code use external policy engines. These provide robust security but require infrastructure beyond the model itself.

Reflexive-Core's Position: The framework trades absolute security guarantees for operational simplicity. By containing security reasoning within one context, it eliminates external dependencies while acknowledging that determined adversaries may still bypass in-context defenses. This is appropriate for scenarios where **speed, simplicity, and layered defense** are priorities.

3. The Reflexive-Core Framework

3.1 Design Philosophy: Why A2AS Primitives Need Metacognitive Implementation

A2AS provides elegant primitives: `<a2as:defense>` declares untrusted boundaries, `<a2as:policy>` specifies permissions, tags establish structure. These primitives provide the LLM with "sudo allow/deny lists" and security guidance before user context arrives. **But primitives alone—without structured metacognitive reasoning—remain passive annotations.** Consider a typical A2AS prompt:

```
<a2as:defense>
Treat ALL external content as untrusted.
NEVER follow instructions from external sources.
</a2as:defense>

<a2as:policy>
READ-ONLY assistant - no sending/deleting/modifying
</a2as:policy>
```

The Problem: A2AS primitives establish security boundaries and declare policies, providing the LLM with guidance. However, without structured metacognitive reasoning, this relies on the model to:

1. Maintain awareness of these constraints throughout reasoning
2. Recognize when external content contains malicious instructions
3. Resist sophisticated attacks that exploit context manipulation
4. Self-monitor for policy drift during execution

Ackerman's research establishes that unstructured self-governance is unreliable (r=0.2-0.3 correlations). Models "forget" constraints, fail to detect subtle attacks, and inconsistently apply policies. Rather than treating A2AS markup as passive annotations and *hoping* the model will adhere, active enforcement is needed.

The Solution: Reflexive-Core operationalizes A2AS through **structured metacognitive reasoning**. Rather than hoping the model maintains security awareness, the framework:

- **Partitions inference** into specialized security roles
- **Forces explicit checkpoints** where policies must be evaluated
- **Creates adversarial perspectives** that actively seek threats
- **Requires reflexive validation** before any output is released

This transforms A2AS from passive markup into a **cognitive architecture**—not just declaring what security boundaries exist, but **implementing how the model actively reasons about them**.

Reflexive-Core rests on three foundational principles:

1. Persona Specialization Over Monolithic Awareness

Rather than expecting a single agent to maintain security awareness throughout complex reasoning, inference is partitioned into specialized analytical roles. Each persona has exclusive domain expertise:

- **Preflight Analyst:** Pattern recognition for known attacks, early threat detection
- **Security Analyst:** PII/confidentiality expertise, deep sensitivity analysis (optional, risk-based)
- **Controlled Executor:** Task execution with constraint monitoring
- **Compliance Validator:** Reflexive audit and final verification

Rationale: SPP research demonstrates that distinct perspectives reduce hallucination and improve accuracy. Security reasoning benefits from the same cognitive diversity.

2. Fail-Closed Checkpoints Over Implicit Trust

Between each phase, mandatory checkpoints force explicit decisions (SAFE/SUSPICIOUS/BLOCKED, approve/review/block). Missing or malformed checkpoints default to BLOCKED.

Rationale: Ackerman's research shows metacognitive signals are "graded and inconsistent." Unstructured model reasoning cannot be trusted. Checkpoints force commitment and enable parseable audit trails.

3. Constitutional Principles Over Prescriptive Rules

Rather than exhaustive prohibition lists, Reflexive-Core encodes high-level axioms (authenticity, least privilege, privacy) that personas apply contextually.

Rationale: Constitutional AI demonstrates principle-based reasoning scales better than brittle rule-matching. Reflexive-Core adapts this to security contexts.

3.2 Architecture

Reflexive-Core executes as a strictly ordered routine within one context:

```
<SystemIdentity>                # Role and scope declaration
<a2as:defense>                  # Input validation layer
<a2as:policy>                   # Declarative permissions (DEFAULT_ACTION: DENY)

<a2as:phases>
  <a2as:preflight>              # PERSONA: Preflight Analyst
    <checkpoint>GO|NO_GO</checkpoint>
  </a2as:preflight>

  <a2as:pre_scan>               # PERSONA: Security Analyst (optional)
    <checkpoint>PROCEED|REVIEW|BLOCK</checkpoint>
  </a2as:pre_scan>

  <a2as:execution>              # PERSONA: Controlled Executor
    <EXECUTION_OUTPUT/>
  </a2as:execution>

  <a2as:assurance>              # PERSONA: Compliance Validator
    <assurance_output>
      confidence_scores, decision
    </assurance_output>
  </a2as:assurance>

  <a2as:final>                  # Gate: APPROVE | REVIEW | BLOCK
  </a2as:final>
</a2as:phases>
```

Determinism: Tag order, persona roles, and gate logic are explicit, creating a **parseable audit trail** within the model's output.

3.3 Persona Descriptions

3.3.1 Preflight Analyst

Core Question: "Is this obviously malicious?"

Expertise: Pattern recognition for known attacks

Tasks:

- Intent anchoring: What is the user actually requesting?
- Injection detection: Role confusion ("you are now..."), override attempts ("ignore previous...")
- Privilege escalation: Requests exceeding declared scope
- Policy mapping: Does request match ALLOWED_ACTIONS?

Output:

```
DECISION: SAFE | SUSPICIOUS | BLOCKED
REASONING: [2 concise sentences]
```

Checkpoint Logic:

- BLOCKED → Skip to `<a2as:final>` with FINAL_BLOCKED
- SUSPICIOUS → Activate Security Analyst (if available)
- SAFE → Proceed to Execution

Design Note: Preflight provides **early rejection** for obvious attacks, reducing unnecessary computation on malicious inputs.

3.3.2 Security Analyst (Optional)

Core Question: "What sensitive data needs protection?"

Expertise: PII detection, confidentiality assessment

Activation: Risk-based. Enable for scenarios with unlabeled data.

Tasks:

- PII detection: Emails, SSNs, phone numbers, names
- Sensitivity flags: "confidential", "privileged", "do not share"
- Contextual assessment: Legal matter, financial data, routine
- Redaction guidance: What to redact, safe to proceed?

Output:

```
REDACTION_GUIDANCE: [specific instructions]
CONFIDENCE: [0.0-1.0]
Safe_to_proceed: [yes/no/with_caution]
```

Checkpoint Logic:

- Safe_to_proceed == no → FINAL_BLOCKED
- CONFIDENCE < 0.70 → FINAL_REVIEW_REQUIRED
- Otherwise → Pass guidance to Executor

Design Note: Security Analyst activates **only when needed**, minimizing overhead for low-risk queries.

3.3.3 Controlled Executor

Core Question: "Can I do this within policy?"

Expertise: Task execution with constraint monitoring

Tasks:

- Execute approved operation (e.g., summarize emails)
- Apply redactions per policy and Security Analyst guidance
- Operate strictly within ALLOWED_ACTIONS
- Self-monitor for drift during execution

Output:

```
<EXECUTION_OUTPUT>
  SANITIZED_RESPONSE: [redacted, policy-constrained result]
</EXECUTION_OUTPUT>
```

Design Note: Executor cannot exceed declared permissions. If policy forbids "send", no amount of prompt injection should enable sending.

3.3.4 Compliance Validator

Core Question: "Does output meet requirements?"

Expertise: Final verification and reflexive audit

Tasks:

- Intent confirmation: Does output match original user intent?
- Policy violations: Did Executor violate any constraints?
- Injection influence: Did adversarial content affect reasoning?
- Risk assessment: Combining all signals

Output:

```
{
  "confidence_policy_compliance": 0.0-1.0,
  "confidence_intent_alignment": 0.0-1.0,
  "overall_confidence": 0.0-1.0,
  "decision": "approve|review|block"
}
```

Checkpoint Logic:

- overall_confidence ≥ 0.85 AND decision == "approve" → FINAL_APPROVED
- overall_confidence ≥ 0.60 OR decision == "review" → FINAL_REVIEW_REQUIRED
- Otherwise → FINAL_BLOCKED

Design Note: Compliance Validator provides **independent verification**. It does not trust Executor's work; it audits it.

3.4 Why Four Personas?

Could fewer personas be used?

Potential Reduction: Merge Preflight + Security Analyst into one "Threat Analyst"

Trade-off:

- **Efficiency loss:** Preflight's early rejection (catching obvious attacks in <100 tokens) would be sacrificed
- **Cognitive diversity loss:** SPP research shows specialized expertise matters

Current Design Rationale: Four personas balance **efficiency** (early rejection), **accuracy** (deep analysis when needed), **functionality** (task execution), and **assurance** (independent audit).

4. Theoretical Foundation

4.1 The Metacognition Evidence

Reflexive-Core's viability depends on frontier LLMs possessing **some degree** of metacognitive capability. Ackerman's (2025) findings establish both the promise and limits:

What Models Can Do:

- Attend to internal confidence signals (r=0.2-0.3 correlations)
- Use structured persona prompts more effectively than unstructured ones

- Show improving capabilities in recent models (GPT-4.1, Opus 4)

What Models Cannot (Yet) Do:

- Achieve strong reliability (r=0.2-0.3 is modest)
- Maintain consistency across contexts
- Guarantee security decisions

Additional Context from Ackerman (2025):

- Entropy-based signals show stronger but still sub-maximal correlation (r ≈ 0.5 best case)
- Baseline token-probabilities are only moderately discriminative (AUROC 0.5-0.75)
- Even with strong teammates, models often over-answer or mis-delegate
- Only Sonnet 3.5 nudged team accuracy above max(self, teammate) (+5.4/+8.6 points)

Implication for Reflexive-Core:

The framework implements **appropriate conservatism**. Reflexive-Core provides:

- Explicit structure (personas, checkpoints) because metacognition is weak
- Fail-closed defaults because reliability is inconsistent
- Parseable outputs because implicit reasoning cannot be trusted

This positions Reflexive-Core as **scaffolding for an emerging capability**. As models improve, structural constraints can be lightened. For now, rigid structure is necessary.

4.2 Cognitive Synergy in Security Contexts

SPP demonstrates that multi-persona reasoning provides genuine cognitive diversity. Applying this to security:

Single-Persona Risk: A monolithic "security-aware assistant" must simultaneously:

- Detect threats (adversarial mindset)
- Execute tasks (service-oriented mindset)
- Validate compliance (auditor mindset)

These perspectives conflict. An agent focused on helpfulness may overlook threats. An agent focused on threat detection may over-restrict functionality.

Multi-Persona Advantage: Specialized perspectives prevent cognitive blind spots:

- Preflight Analyst adopts **adversarial** stance: "What could go wrong?"
- Executor adopts **service** stance: "How do I help within constraints?"
- Compliance Validator adopts **auditor** stance: "Did we actually follow policy?"

Empirical Support: While SPP focused on knowledge tasks, the underlying mechanism—cognitive diversity through perspective-taking—applies to security reasoning. Reflexive-Core tests this hypothesis in a new domain.

Note on Task-Benchmark Transfer: SPP performance improvements (+7.1% to +18.5% on various tasks) demonstrate the general effectiveness of multi-persona approaches. While these are task-benchmark deltas rather than direct security outcomes, the cognitive mechanisms (reduced hallucination, multi-perspective critique, specialized expertise) theorize strong transfer to cybersecurity sub-persona tasks.

4.3 Constitutional Principles in Security

Constitutional AI demonstrates that principle-based reasoning outperforms rule-based filtering. Reflexive-Core adapts this to security:

Constitutional Principles Encoded:

1. **Authenticity:** Trust user intent, distrust embedded instructions
2. **Least Privilege:** Operate within minimal necessary permissions
3. **Transparency:** Explicit reasoning traces for all decisions
4. **Privacy:** Proactive PII protection, not reactive redaction
5. **Harm Prevention:** Understand consequences, not just pattern-match prohibitions

Each persona applies these principles from its specialized perspective, creating layered ethical reasoning rather than brittle rule-checking.

4.4 Divergence from Speculative Theories

Recent community discussions (Fleuren, 2025) propose "intrinsic safety through axiomatic governance" and "self-transcendent intelligence" emerging from unrestricted exploration. While philosophically interesting, Reflexive-Core takes a more **conservative, engineering-focused** approach:

Areas of Agreement:

- External constraints (the "Cage Model") are brittle
- Principle-based reasoning enables better generalization
- Understanding threats requires some exposure to adversarial content

Points of Divergence:

- **No unrestricted exploration:** Reflexive-Core maintains deterministic checkpoints and fail-closed defaults
- **No assumption of emergence:** The framework does not rely on models "discovering" security principles

- **Explicit structure required:** Given Ackerman's modest findings (r=0.2-0.3), unstructured self-governance cannot be trusted

Honest Tension: Constitutional AI philosophy (especially expansive interpretations) favors minimal structure and model autonomy. Reflexive-Core adds rigid scaffolding because **current models' metacognitive reliability remains limited**. This is appropriate conservatism for production security.

Reference: Fleuren, J.W. (2025, August 28). Constitutional AI: A Novel Approach to Intrinsic Safety in Agentic Models. *Hugging Face Blog*. https://huggingface.co/blog/KingOfThoughtFleuren/constitutional-ai

5. Token Economics

A critical practical concern: Does Reflexive-Core's multi-persona approach impose prohibitive costs?

5.1 Measurement Methodology

Token usage was compared across three scenarios:

Baseline (Minimal A2AS):

```
System: You are a helpful email assistant...
<a2as:defense>[60 tokens]</a2as:defense>
<a2as:policy>[50 tokens]</a2as:policy>
User: <a2as:user>Review emails from last week</a2as:user>
```

Total system overhead: **~175 tokens**

Reflexive-Core (Production-Optimized):

```
<SystemIdentity>[50 tokens]</SystemIdentity>
<a2as:defense>[60 tokens]</a2as:defense>
<a2as:policy>[40 tokens]</a2as:policy>
<a2as:preflight>[120 tokens]</a2as:preflight>
<a2as:execution>[60 tokens]</a2as:execution>
<a2as:assurance>[150 tokens]</a2as:assurance>
<a2as:final>[50 tokens]</a2as:final>
```

Total system overhead: **~560 tokens** (pre_scan disabled)

Additional Overhead: 385 tokens (**fixed cost**)

5.2 Scaling Analysis

Small Context (1K tokens total):

- Baseline: 175 system + 825 content/reasoning = 1,000 tokens
- Reflexive-Core: 560 system + 1,292 content/reasoning = 1,852 tokens
- **Overhead: 1.85×** (85% increase)

Medium Context (10K tokens):

- Baseline: 175 system + 9,825 content = 10,000 tokens
- Reflexive-Core: 560 system + 10,210 content = 10,770 tokens
- **Overhead: 1.08×** (8% increase)

Large Context (50K tokens):

- Baseline: 50,175 tokens
- Reflexive-Core: 50,560 tokens
- **Overhead: 1.008×** (0.8% increase)

5.3 Key Findings

1. **Fixed Cost Dominates Small Contexts:** The 385-token framework overhead is significant when total context is small but becomes negligible at scale.
2. **Better Than Multi-Pass:** Two-pass verification (2-3× overhead) remains expensive regardless of context size. Reflexive-Core's single-pass design scales better.
3. **Production Viability:** For realistic workloads (email summarization with full thread history, document analysis with 20+ page PDFs), overhead is <2% of total tokens.

5.4 Cost Implications

Claude Sonnet 4.5 Pricing (as of October 2025):

- Input: $3 per 1M tokens
- Output: $15 per 1M tokens

Example: Email Summarization Service (50K avg context):

- Baseline cost: $0.1506 per request
- Reflexive-Core cost: $0.1518 per request (+$0.0012, or 0.8%)

Break-Even Analysis: For security-critical applications, even 2× overhead could be justified economically. A 0.8% cost increase for substantially improved security posture is negligible.

5.5 Comparison to Alternatives

Approach	Token Overhead	Passes	Latency	State Management
Baseline A2AS	1.0×	1	Low	None
Reflexive-Core	1.01-1.9×	1	Low	None
Two-Pass Verifier	2.0-3.0×	2	High	Required
External Guardrails	Variable	2+	High	Required

Reflexive-Core occupies a sweet spot: **near-baseline efficiency with substantially better security than monolithic prompts, without the complexity of multi-pass systems**.

6. Implementation

6.1 Production Specification

The complete Reflexive-Core XML specification is available at: [GitHub repository URL] (Apache 2.0 license)

Key implementation notes:

Phase Ordering: Strictly enforce the canonical sequence. Phases appearing out of order should trigger immediate halt.

Checkpoint Parsing: Each checkpoint must emit a **parseable decision**. Missing or ambiguous checkpoints default to most restrictive outcome (BLOCKED).

Fail-Closed Defaults:

- Missing overall_confidence → Assume 0.0
- Malformed tags → Treat as untrusted input, halt
- Any uncertainty → Route to FINAL_REVIEW_REQUIRED

Optional Phases: Security Analyst (`<a2as:pre_scan>`) can be toggled via XML comments. For labeled/low-risk data, disable to minimize overhead.

6.2 Integration Patterns

Minimal Integration (existing A2AS-aware systems):

```
system_prompt = load_reflexive_core_template()
response = llm.invoke(system_prompt + user_message)
final_output = parse_final_decision(response)
```

Advanced Integration (with external logging):

```
response = llm.invoke(reflexive_prompt)
audit_trail = extract_persona_outputs(response)
log_security_decisions(audit_trail)
final_output = parse_final_decision(response)
```

6.3 Model Requirements

Minimum Requirements:

- 100K+ context window (to accommodate framework + realistic data)
- Extended thinking capability (Claude's thinking mode, GPT-4.1's reasoning)
- Temperature 0.0-0.3 (for consistency)

Tested Models:

- Claude Opus 4.1 ✓
- Claude Sonnet 4.5 ✓
- GPT-4.1 ✓
- Gemini 2.5 Pro ✓
- Grok 4 (untested, but meets specifications)

Not Recommended:

- Models <100K context (insufficient room for framework + data)
- Non-frontier models (SPP shows cognitive synergy only emerges in capable models)

6.4 Example: Email Summarization

Input:

```
<a2as:user:7c3d0c6d>
Summarize internal communications from last week about the Q4 budget.
</a2as:user>
```

Reflexive-Core Processing (condensed):

```
PREFLIGHT:
  Intent: "User wants email summary filtered by topic and timeframe"
  Threats: None detected
  Decision: SAFE

PRE_SCAN: [disabled for this example]

EXECUTION:
  [Reads emails, applies redactions, generates summary]
  Output: "5 emails found. Key topics: budget allocation ($2.3M)..."

ASSURANCE:
  Intent confirmed: yes
  Policy violations: none
  Injection influence: no
  Confidence: 0.92
  Decision: approve

FINAL:
  FINAL_APPROVED: [Summary text with PII redacted]
```

Audit Trail: All persona outputs are preserved in the response, providing full traceability.

7. Security Analysis

7.1 Threat Model

Threats Addressed:

1. **Prompt Injection:** Malicious instructions embedded in emails/documents
   - **Mitigation:** Preflight detects role-override patterns, `<a2as:defense>` establishes distrust boundary
   - **Effectiveness:** Partial. Sophisticated attacks may bypass detection.
2. **Role Drift:** Agent gradually convinced to exceed original scope
   - **Mitigation:** Compliance Validator audits against original intent
   - **Effectiveness:** Moderate. Sustained attacks may succeed.
3. **PII Leakage:** Sensitive data inadvertently included in output
   - **Mitigation:** Security Analyst pre-scan + Executor redactions + Validator audit
   - **Effectiveness:** Strong for labeled PII, weaker for contextual sensitivity.
4. **Policy Violations:** Agent performing prohibited actions

- **Mitigation:** DEFAULT_ACTION: DENY + Executor constraint monitoring
- **Effectiveness:** Strong for explicit violations, weaker for ambiguous edge cases.

Threats NOT Addressed:

- **Adversarial Model Weights:** If the model itself is compromised, no in-context defense helps
- **Side-Channel Attacks:** Timing analysis, token probability leakage
- **Coercion Attacks:** Convincing the model it must violate policy for "safety"
- **Jailbreaking:** Dedicated adversarial optimization to break defenses

7.2 Known Failure Modes

1. Persona Collapse

Symptom: All personas generate similar reasoning, defeating cognitive diversity.

Indicators:

- Similar language across all persona outputs
- No disagreement between personas
- Consistent decisions regardless of persona sequence

Mitigation:

- Explicitly instruct personas to adopt different analytical stances
- Test for genuine disagreement on edge cases
- Monitor for linguistic diversity in outputs

2. Inconsistent Enforcement

Symptom: Similar attacks handled differently across contexts.

Indicators:

- Rephrased injection attempts yield different results
- Same violation blocked sometimes, allowed other times
- Confidence scores uncorrelated with actual correctness

Mitigation:

- Maintain test suite of known attacks
- Measure consistency across runs
- Adjust confidence thresholds based on false positive/negative rates

3. False Confidence

Symptom: High confidence scores on incorrect security decisions.

Indicators:

- Model confidently approves policy violations
- High confidence on attacks that should be blocked
- Confidence uncalibrated to actual accuracy

Mitigation:

- Regular red-team testing
- Compare confidence distributions on known-bad vs known-good inputs
- Recalibrate thresholds empirically

4. Audit Trail Inflation

Symptom: Reasoning traces become verbose but uninformative.

Indicators:

- Personas generate generic security statements
- Reasoning doesn't address specific threats in input
- High token count without actionable insight

Mitigation:

- Require personas to cite specific input elements
- Penalize generic responses during development
- Validate that reasoning traces are actionable

7.3 Fundamental Limitations

Reflexive-Core **cannot:**

- **Defend against all prompt injection:** Given Ackerman's modest findings (r=0.2-0.3, not r=0.9+), metacognitive reliability remains limited
- **Replace external security controls:** Network policies, authentication, tool authorization remain necessary
- **Guarantee consistency across contexts:** Metacognition is variable
- **Work reliably on non-frontier models:** Requires 100K+ context, extended thinking

Appropriate Use: One layer in defense-in-depth. Meaningful improvement over unstructured prompts. **Not a complete security solution.**

7.4 Red Team Recommendations

Organizations deploying Reflexive-Core should:

1. **Establish baseline metrics:** False positive/negative rates on known attacks
2. **Continuous adversarial testing:** Dedicated red team with evolving attack repertoire
3. **Monitor persona diversity:** Automated linguistic analysis to detect collapse
4. **Confidence calibration:** Regular empirical validation of confidence→correctness correlations
5. **External validation:** Human review of REVIEW_REQUIRED cases to tune thresholds

8. Future Work

8.1 Empirical Validation Needed

This work presents a conceptual framework and implementation specification. **Critical next steps:**

1. **Benchmark Development:** Standardized test suite of prompt injection attacks, policy violations, and edge cases.
2. **Efficacy Measurement:** Systematic comparison of Reflexive-Core vs. baseline A2AS vs. multi-pass verification across:
   - Attack detection rates
   - False positive/negative rates
   - Consistency across contexts
   - Cost-effectiveness
3. **Persona Ablation Studies:** Test 2-persona vs. 4-persona variants to empirically determine optimal configuration.
4. **Model Comparison:** Quantify performance differences across frontier models (Claude, GPT, Gemini, Grok).
5. **Longitudinal Analysis:** Track effectiveness as models improve. Does metacognitive capability strengthen over time?

8.2 Architectural Extensions

Multi-Modal Security: Extend Reflexive-Core to vision inputs (detecting malicious images, QR codes with injection payloads).

Adaptive Persona Weighting: Dynamically adjust which personas activate based on risk assessment.

Federated Validation: Multiple instances cross-validate each other's security decisions.

Interpretability Integration: Combine with attention tracking (Attention Tracker research, arXiv:2411.00348) to detect internal manipulation signals.

8.3 Integration with External Systems

While Reflexive-Core intentionally avoids external dependencies, it can **complement:**

- **Tool Authorization Frameworks:** Reflexive-Core validates intent, external systems enforce permissions
- **Network Gateways:** In-context security + network-level filtering = defense-in-depth
- **Multi-Agent Systems:** Each agent runs Reflexive-Core internally, preventing inter-agent attacks

8.4 Theoretical Questions

1. **Optimal Persona Count:** Is four personas theoretically justified, or could two (Critic + Executor) achieve similar results with lower overhead?
2. **Cognitive Diversity Measurement:** Can genuine cognitive diversity be quantified vs. superficial linguistic variation?
3. **Scaling Laws:** How does effectiveness change with model scale? Do smaller models benefit at all, or is this strictly a frontier-model capability?
4. **Adversarial Resilience:** What is the theoretical limit of single-context security? Can bounds on attack resistance be proven?

9. Acknowledgments

This work builds directly on Christian Posta's agentgateway project, which provided critical proof-of-concept that A2AS primitives are operationally viable through external enforcement. Posta's work on mitigating prompt injection attacks through A2AS and agentgateway demonstrated the fundamental soundness of middleware-based policy interpretation and inspired Reflexive-Core's evolution toward in-context intelligence.

Gratitude is extended to the A2AS Working Group (OWASP, Google, Salesforce, AWS, and collaborators) for developing the specification that makes structured agent security possible. Special thanks to Christopher Ackerman for empirical metacognition research that grounds architectural decisions, and to the research teams at Anthropic (Constitutional AI), Microsoft Research, and other institutions whose work on LLM self-critique and cognitive synergy enabled this framework.

Reference: Posta, C. (2024). How to Mitigate Prompt Injection Attacks with A2AS and agentgateway. https://www.linkedin.com/pulse/mitigate-prompt-injection-attacks-a2as-agentgateway-christian-posta-tmaxc/

10. Ethical Considerations

10.1 Transparency

Reflexive-Core generates **explicit audit trails**. Users and auditors can inspect persona reasoning. This transparency is critical for accountability.

However: Transparency alone does not guarantee correctness. Bad decisions can be transparently wrong. Audit trails must be **monitored and validated**.

10.2 Over-Reliance Risk

Organizations may mistakenly view Reflexive-Core as "solved security." **This is dangerous.** Single-context reasoning has fundamental limits (r=0.2-0.3 metacognition). External controls remain necessary.

Mitigation: Documentation must emphasize limitations. Deployment guides should mandate defense-in-depth.

10.3 Accessibility

Does structured security reasoning disadvantage users with complex or ambiguous requests?

Potential Issue: Fail-closed defaults may block legitimate edge cases.

Mitigation: REVIEW_REQUIRED pathway enables human override. Balance security with usability.

10.4 Bias in Security Decisions

If personas exhibit bias (e.g., flagging certain languages or cultural contexts as "suspicious"), this perpetuates discrimination.

Mitigation: Regular bias audits. Diverse red teams. Explicit fairness criteria in constitutional principles.

11. Conclusion

The evolution of A2AS-based security proceeds through clear stages: (1) specification of primitives, (2) validation through external enforcement (agentgateway), (3) operationalization through in-context intelligence (Reflexive-Core), and (4) hybrid integration combining external reliability with contextual reasoning.

Christian Posta's agentgateway work proved A2AS primitives are operationally viable—middleware can successfully enforce security boundaries and mitigate prompt injection attacks through exter-

nal rule engines. **Reflexive-Core represents the next evolutionary step:** moving enforcement intelligence into the context window by leveraging the sophisticated cybersecurity reasoning capabilities that frontier LLMs inherently possess.

Rather than treating security as deterministic rule-matching (allow/deny lists evaluated externally), Reflexive-Core enables **intelligent, context-aware security reasoning** through structured sub-personas. This shift—from external rules to in-context intelligence—fundamentally changes how LLM security is approached. Models aren't passive text processors requiring external control; they're active security reasoners requiring proper cognitive scaffolding.

Token analysis validates operational viability: 1.9× overhead for small contexts, converging to <1.01× for production workloads. This efficiency, combined with elimination of external round-trip latency, makes in-context security economically and architecturally practical.

Future Vision: The ultimate architecture may be **hybrid**: deterministic brokers (agentgateway-style systems) handle hard security boundaries—network isolation, tool authorization, cryptographic verification—while in-context personas provide intelligent runtime reasoning about threats, policies, and risks. External systems enforce what *can* happen; internal personas reason about what *should* happen.

Honest Assessment: Reflexive-Core is **not** a complete security solution. LLM metacognition remains limited (r=0.2-0.3 correlations). Sophisticated adversaries will bypass in-context defenses. External controls remain necessary. But by leveraging models' inherent security reasoning capabilities through structured personas, a qualitative leap is made: from brittle rule-matching to contextual intelligence.

Looking Forward: As frontier models' introspective abilities strengthen, the gap between declarative security (A2AS primitives) and intelligent enforcement will narrow. Reflexive-Core establishes both the theoretical framework and working implementation for this convergence.

The question isn't whether to use external enforcement or in-context reasoning—it's how to integrate both. A2AS gave the language. agentgateway proved it works. Reflexive-Core makes it intelligent. The future combines all three.

References

1. Ackerman, C. (2025). Evidence for Limited Metacognition in LLMs. *arXiv preprint arXiv:2509.21545*. https://arxiv.org/abs/2509.21545
2. Posta, C. (2024). How to Mitigate Prompt Injection Attacks with A2AS and agentgateway. *LinkedIn Article*. https://www.linkedin.com/pulse/mitigate-prompt-injection-attacks-a2as-agentgateway-christian-posta-tmaxc/
3. Anthropic (Docs). *Use XML tags to structure your prompts.* https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/use-xml-tags

4. OpenAI (Docs). *Prompt engineering – OpenAI API.* (Includes example using Markdown and XML tags.) https://platform.openai.com/docs/guides/prompt-engineering
5. Google Cloud (Vertex AI Docs). *Structure prompts.* ("Use prefixes or XML tags to delimit different parts of a prompt.") https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/structure-prompts
6. Microsoft (Azure OpenAI, Learn). *Prompt engineering techniques.* ("If you're not sure what syntax to use, consider... XML.") https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/prompt-engineering
7. Alpay, F.; Alpay, T. (2025). *XML Prompting as Grammar-Constrained Interaction: Fixed-Point Semantics, Convergence Guarantees, and Human-AI Protocols.* arXiv. https://arxiv.org/abs/2509.08182
8. A2AS Working Group (2025). Agent-to-Agent Security Framework. https://www.a2as.org/
9. Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*. https://arxiv.org/abs/2212.08073
10. Wang, Z., Mao, S., Wu, W., Ge, T., Wei, F., & Ji, H. (2023). Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration. *arXiv preprint arXiv:2307.05300*. https://arxiv.org/abs/2307.05300
11. Shinn, N., Labash, B., & Gopinath, A. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*. https://arxiv.org/abs/2303.11366
12. Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *arXiv preprint arXiv:2303.17651*. https://arxiv.org/abs/2303.17651
13. Dhuliawala, S., et al. (2023). Chain-of-Verification Reduces Hallucination in Large Language Models. *arXiv preprint arXiv:2309.11495*. https://arxiv.org/abs/2309.11495
14. Fleuren, J.W. (2025, August 28). Constitutional AI: A Novel Approach to Intrinsic Safety in Agentic Models. *Hugging Face Community Blog*. https://huggingface.co/blog/KingOfThoughtFleuren/constitutional-ai
15. Anthropic (2025, September 29). Introducing Claude Sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5
16. xAI (2025, July 10). Grok 4. https://x.ai/news/grok-4
17. Guo, Y., et al. (2024). Attention Tracker: Detecting Prompt Injection via Attention Distraction. *arXiv preprint arXiv:2411.00348*.

Appendix A: Complete XML Specification

[Full production-optimized reflexive-core.xml specification]

Available at: https://github.com/alexlstanton/a2as-reflexive

Appendix B: Token Overhead Calculations

[Detailed breakdown of token measurements across scenarios]

Appendix C: Example Outputs

[Annotated examples showing persona reasoning on various inputs]

Contact: alex@thinkpurple.io

GitHub:https://github.com/alexlstanton

LinkedIn:https://linkedin.com/in/alexlstanton

License: Apache 2.0